EE 542: Lab Assignment 8

Creating Custom Chatbot with Large Language Model and Embeddings

Instructor: Young H. Cho, Ph.D.

Prerequisites

1. OS: Linux, macOS, or Windows
2. MySQL 8+
3. Python 3.10+ and pip
4. Node.js + npm if you plan to use the bubblewrap CLI
5. Android Studio Flamingo+ (or newer), JDK 17, Android SDK

Part 1: Web Application

I. Templates

1. Download the ZIP file for this Assignment
2. Review the README file and follow the instructions

II. MySQL: Install & Database Setup

1. Install MySQL Server (consult your OS package manager or installer).
2. Start MySQL and secure it (set root password).
3. Create database and user (or run sql/schema.sql):

```
CREATE DATABASE groupchat CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'chatuser'@'localhost' IDENTIFIED BY 'chatpass';
GRANT ALL PRIVILEGES ON groupchat.* TO 'chatuser'@'localhost';
FLUSH PRIVILEGES;
```

III. Backend (FastAPI) Setup

```
cd backend
python -m venv .venv
source .venv/bin/activate   (Windows: .venv\Scripts\activate)
pip install -r requirements.txt
```

1. Copy .env.example to `../.env` and edit values (DB DSN, JWT secret, LLM endpoint)
2. Run the server: `uvicorn app:app --host 0.0.0.0 --port 8000`
3. Open `http://localhost:8000` in a browser.
4. Sign up, then log in. Messages with a "?" trigger the LLM bot.

IV. Configuration (`.env`)

1. Edit .env using an editor like gedit/nano/vim.
2. Then copy paste the bellow lines and save the file.

```
DATABASE_URL=mysql+asyncmy://chatuser:chatpass@localhost:3306/groupchat
JWT_SECRET=<replace with a long random string>
JWT_EXPIRE_MINUTES=43200
LLM_API_BASE=http://localhost:8001/v1
```

```
LLM_MODEL=llama-3-8b-instruct
LLM_API_KEY=
APP_HOST=0.0.0.0
APP_PORT=8000
```

V. Self-Hosted Llama 3 Endpoint (Option A)

1. Build llama.cpp:

```
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp
cmake -B build
cmake --build build --config Release
```

2. Obtain Llama 3 weights from Meta/Hugging Face. Accept the license. Download the model.
3. One option: https://huggingface.co/bartowski/Meta-Llama-3.1-70B-Instruct-GGUF
4. (If needed) Convert to GGUF using llama.cpp's conversion tools (see llama.cpp docs).
5. Start the HTTP server (OpenAI-compatible):

```
cd llama.cpp/build/bin
./llama-server -m /path/to/model.gguf --host 0.0.0.0 --port 8001 --ctx-size
4096
```

6. Optionally enforce an API key:

```
# ./server -m /path/to/model.gguf --host 0.0.0.0 --port 8001 --api-key MYKEY
```

7. In the chat app .env set:

```
LLM_API_BASE=http://localhost:8001/v1
LLM_MODEL=<your model name or leave as placeholder>
LLM_API_KEY=MYKEY   (only if you set --api-key)
```

8. Restart the FastAPI app and test by sending a question in chat.

VI. Self-Hosted Llama 3 Endpoint (Option B)

1. Install vLLM in a Python environment with CUDA (if using GPU).
2. Launch OpenAI-compatible server, e.g.:

```
python -m vllm.entrypoints.openai.api_server --model <your-llama3-model>
--host 0.0.0.0 --port 8001
```

3. Configure the chat app .env similarly to Option A.

VII. Frontend

The frontend is plain HTML/CSS/JS served by FastAPI as static files. It connects via REST for login and message posting, and via WebSocket for real-time message broadcast.

Part 2: Mobile Application Wrapper Interface

I. Android Studio

1. Go to https://developer.android.com/studio to download and install Android Studio
2. While this source has been tested using Android Studio Flamingo, it is expected that the latest version of Android Studio would be able to compile the code.

II. Android Trusted Web Activity (TWA) Application

1. Open the Android project in twa_android_src on Android Studio.
2. Change applicationId/package as needed in app/build.gradle and manifest.
3. Set your deployed web URL in res/values/strings.xml as <string name="defaultUrl">https://YOUR_DOMAIN/</string>
4. Build & run on a device.

III. Asset Links (Required For Full TWA Experience)

1. Generate or use your signing key. Get the SHA-256 fingerprint:

```
keytool -list -v -keystore your_keystore.jks -alias your_alias -storepass <storepass>
-keypass <keypass>
```

2. Copy the SHA-256 fingerprint.
3. On your web server, host `/.well-known/assetlinks.json`. A template is included in the web app at `frontend/.well-known/assetlinks.json`. Replace REPLACE_WITH_YOUR_SHA256_FINGERPRINT.
4. The package name in `assetlinks.json` must match your Android app ApplicationID.
5. After deploying `assetlinks.json`, re-install your signed app and launch. Chrome should open your site as a TWA without URL bar.

IV. Android App

Compile, install, and test your Android App (APK) for your Website on either an Android phone or an Android Emulator

a) LDPlayer: https://www.ldplayer.net
b) BlueStacks: https://www.bluestacks.com/bluestacks-5.html
c) KoPlayer: https://koplayerpc.com/

Part 3: Content Embeddings and LLM

I. Website Design for Data Collection

1. Download the zip file for Part 3
2. Design a website that you can use to upload PDFs, text, and other documents, then convert and divide them into chunks so that you can vectorize/embed them.

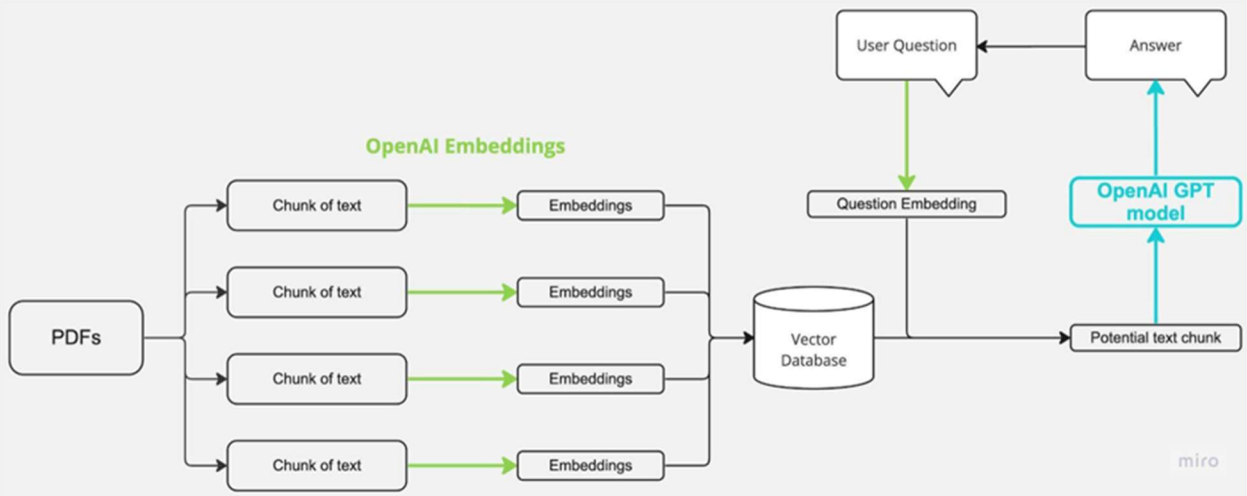
3. Provide a sidebar option where the user can upload single or multiple PDF documents for analysis.

II. Data Processing and Storage.

1. **PDF Text Extraction**
   Your task is to write a Python script to iterate over the PDF in the folder, extract the information from the PDF, and store it in your database tables.
2. **Get Text Chunks**
   Process the extracted data into chunks of size 500. Use the ***CharacterTextSplitter*** provided by langChain to split the text. Read about the different parameters the function takes and set appropriate values for those parameters to split the extracted data into chunks efficiently.
3. **Create Vector Datastore**
   Write a function that would take the chunks generated in the above step as input and use the OpenAI Embeddings to create word embeddings for those chunks and store them in a vector database.
4. **Create a Conversation Chain**
   Write a function that would take an LLM model, and the vector store created above as input and create a conversation chain to understand user questions and generate responses based on its knowledge from the vector database.
   (Take a look at the ConversationMemoryBuffer function provided by langchain for writing this function)

III. Driver Function

1. Write a driver function that calls the above functions and then gets questions from the user, runs it through the conversation chain to get a response, prints the response on the screen, and then asks the user for the next question. The user must type in 'exit' to quit the program. Use OpenAI to create conversation chains.
2. Take the converted text contents into Embeddings using OpenAI's embeddings library and store them in a vector database. The LLM model should generate custom answer to questions only based on the closest matching context to the question found in the databse.
3. Summary
   a) Collect: Convert PDF into Text document suitable for embeddings and GPT
   b) Chunk: The text document will need to be split into short, mostly self-contained sections to be embedded
   c) Embed: Each section is embedded with the OpenAI API
   d) Store: Embeddings are saved (for large datasets, use a vector database)
   e) Given a user question, generate an embedding for the query from the OpenAI API
   f) Using the embeddings, rank the text sections by relevance to the query.
   g) Insert the question and the most relevant sections into a message to GPT.
   h) Return GPT's answer.



Part 4: Custom Content Chatbot App

Now that you have some quick and dirty experience with the LLM Chatbot, Website File Upload/Download interface, Embeddings and Databases, and Android App Development, you should be able to combine everything together and sprinkle it with domain knowledge to create a custom Chatbot Android App that answers questions on a specific area of interest with your own knowledge base.

Your team should take some time to think and explore a topic that is popular with a specific group of people. Selectively collect and curate a sufficiently large amount of data for that topic with a consistent view. Embed that data to build a database. Then, incorporate the embeddings along with the LLM into your chatbot's Web interface and make the necessary changes to the website and App to make it work with your Android App.

Create a useful App to answer questions about a specific topic of interest based on your curated data for the benefit of that interest group. One example might be an App that answers questions about immigration laws. Another application might be for a food recipe given a certain ingredient that you have in your refrigerator. Be creative and professional in this final part so that the system can be potentially useful in a real-world situation.

Part 5: Custom Embedding

There are many other ways to vectorize your context chunks. One effective way to encapsulate the meaning of chunks of text is to vectorize based on the normalized histogram of word groups. Therefore, this part requires you to re-embed your data chunks using this technique.

- Find and install Word2Vec
- Use Word2Vec to generate word vectors of an N-dimension for each word in your data.
- Write custom code to cluster all of the words into K clusters.
- Determine the centroid vector for each cluster.

- Build a normalized K-dimensional histogram vector for each context chunk of data (Take N-dimensional vector of the word in each chunk and bin it into an entry of a K-dimensional histogram vector based on the distance between centroid vectors and the word vector. Then normalize the histogram vector by dividing the K-dimensional vector by the total number of words in the chunk.)
- Try different methods to improve the result of the embedding.

Deliverables

In the Video Log, demonstrate and describe all of the following

1. Source code
2. Actual examples of contextual data and their embedding and how it is used in RAG system
3. Word histogram embedding method and your improvements
4. Working Website and Android App